# FOM Hochschule für Oekonomie & Management

Hochschulzentrum Köln

## Seminararbeit

im Studiengang KI & Business Analytics

zur Erlangung des Grades eines

Master of Science (M. Sc.)

über das Thema

## Versionierung von Datenmodellen

von

Florian Stevener

| | |
|---|---|
| Betreuer | Prof. Dr.-Ing. Peter Steininger |
| Matrikelnummer | 839237 |
| Abgabedatum | 24.05.2026 |

# Inhaltsverzeichnis

Abbildungsverzeichnis III

Quelltexte IV

1 Einleitung 1

2 Theoretische Grundlagen 2

2.1 Datenbankschema . . . . . 2

2.1.1 Tabelle . . . . . 5

2.1.2 Quelltext . . . . . 7

2.1.3 LaTeX-Befehle . . . . . 8

2.2 Schemamodifikation . . . . . 8

2.3 Schema-Evolution . . . . . 10

2.4 Schema-Versionierung . . . . . 11

3 Methoden der Schema-Versionierung 12

3.1 Dimensionen der Schema-Versionierung . . . . . 13

3.1.1 Partielle und vollständige Schema-Versionierung . . . . . 13

3.1.2 Temporale Schema-Versionierung . . . . . 14

3.1.3 Speicherkonzept . . . . . 18

3.1.4 Interaktion zwischen Instanz und Schema . . . . . 19

3.2 DBMS Support und Tooling . . . . . 22

3.3 Ausblick . . . . . 24

Literaturverzeichnis 25

Internetquellen 25

# Abbildungsverzeichnis


1 ANSI-SPARC-Architektur . . . . . 3
2 Terminal . . . . . 5
3 Zustand vor der Schemamodifikation . . . . . 9
4 Zustand nach der Schemamodifikation . . . . . 10
5 Zustand nach der Schema-Evolution . . . . . 11
6 Zustand nach der Schema-Versionierung . . . . . 12
7 Partielle Schema-Versionierung . . . . . 13
8 Vollständige Schema-Versionierung . . . . . 14
9 Testpyramide . . . . . 23

# Quelltexte

1 Writing Templates . . . . . 8
2 DDL-Statements . . . . . 9

# 1 Einleitung

Änderungen an Datenmodellen treten in datengetriebenen Informationssystemen regelmäßig auf und sind als natürlicher Bestandteil ihrer Weiterentwicklung zu verstehen. Ursachen hierfür liegen unter anderem in sich verändernden Nutzeranforderungen, der Anpassung an neue Geschäftslogiken oder in externen Rahmenbedingungen wie gesetzlichen Vorgaben. Diese strukturellen Anpassungen stellen besondere Anforderungen an die Verwaltung von Datenbeständen, da bestehende Daten und Anwendungen weiterhin funktionsfähig bleiben müssen.

Ein zentraler Ansatz zur Bewältigung dieser Problematik ist die Schema-Versionierung. Dieses Konzept sieht vor, verschiedene Versionen eines Datenbankschemas parallel zu verwalten und die zugehörigen Daten konsistent diesen Versionen zuzuordnen. Dadurch können sowohl historische Datenbestände als auch ältere Anwendungen weiterhin genutzt werden, ohne dass Informationen verloren gehen oder unmittelbare Migrationen erforderlich sind. Schema-Versionierung wurde sowohl für klassische relationale Datenbanksysteme als auch für neuere Datenbankparadigmen umfassend untersucht. Obwohl eine Vielzahl wissenschaftlicher Arbeiten zu diesem Thema existiert, ist die praktische Unterstützung in Datenbanksystemen bislang nur eingeschränkt ausgeprägt.

Vor diesem Hintergrund bietet die vorliegende Seminararbeit eine Einführung in die grundlegenden Konzepte der Schema-Versionierung, wie sie in der wissenschaftlichen Literatur behandelt werden. Der Fokus der Arbeit liegt ausschließlich auf der Versionierung von Datenbankschemata in relationalen Datenbanken.

Methodisch basiert die Arbeit auf einer Literaturanalyse einschlägiger Fachquellen, eigene empirische Erhebungen werden nicht durchgeführt.

Die Arbeit ist wie folgt aufgebaut: Kapitel 2 definiert zunächst die zentralen Begriffe *Datenbankschema*, *Schemamodifikation*, *Schema-Evolution* und *Schema-Versionierung* und grenzt diese voneinander ab. Kapitel 3 behandelt anschließend grundlegende Konzepte der Schema-Versionierung, wie sie in der wissenschaftlichen Literatur beschrieben werden und stellt ausgewählte Softwarelösungen vor. Abschließend erfolgt ein kurzer Ausblick.

# 2 Theoretische Grundlagen

Da der Begriff der Schema-Versionierung bislang nicht einheitlich definiert ist und sowohl in der wissenschaftlichen Literatur als auch in der Praxis unterschiedliche Auffassungen darüber bestehen, wird in diesem Kapitel eine für diese Arbeit gültige Begriffsbestimmung auf Grundlage etablierter wissenschaftlicher Definitionen vorgenommen. Darüber hinaus werden verwandte Konzepte erläutert, die der Schema-Versionierung inhaltlich nahe stehen und daher häufig mit ihr verwechselt oder vermischt werden.

## 2.1 Datenbankschema

Das *Datenbankschema* definiert die Struktur der Datenbank. Es ist eine strukturierte Menge von Metadaten, wird mit der SQL Data Definition Language (DDL) erzeugt und im Systemkatalog des *Datenbankmanagementsystems* (kurz *DBMS*) gespeichert. Im Folgenden wird der Begriff *Schema* synonym für das Datenbankschema verwendet. Schemata sind für die Abfrage, Aktualisierung, Integration und Konvertierung von Daten erforderlich. Der Begriff Schema wird in der Literatur nicht einheitlich verwendet. Die ANSI-SPARC-Architektur definiert Schemata auf drei verschiedenen Abstraktionsebenen: das *interne Schema*, das *konzeptionelle Schema* und das *externe Schema* (vgl. Abb. 1).

Das *interne Schema* definiert die physische Struktur der Datenbank. Es enthält vollständige Informationen über die physische Speicherung der Daten auf dem Datenträger.

Das *konzeptionelle Schema* definiert die logische Struktur der Datenbank. Es abstrahiert die Details der physischen Speicherung und bezieht sich ausschließlich auf das konzeptionelle Design der Datenbank. Es beschreibt Relationen, Attribute, Datentypen, Beziehungen zwischen Entitäten und Integritätsbedingungen. Das konzeptionelle Schema legt damit fest, wie die Daten in der Datenbank angeordnet sind und in welcher Beziehung sie zueinander stehen.

Das *externe Schema* definiert die benutzerspezifische Sicht auf die Datenbank. Es legt fest, auf welche Teilmenge des konzeptionellen Schemas ein Benutzer zugreifen darf. Für verschiedene Benutzergruppen können unterschiedliche externe Schemata existieren.

Die Abstraktionsebenen der ANSI-SPARC-Architektur garantieren Datenunabhängigkeit. Schemaänderungen auf einer Ebene des Datenbanksystems haben damit keine Auswirkungen auf das Schema der nächsthöheren Ebene. Es wird zwischen *physischer Datenunabhängigkeit* und *logischer Datenunabhängigkeit* differenziert. Physische Datenunabhängigkeit beschreibt die Fähigkeit, Änderungen am internen Schema eines Datenbanksystems vorzunehmen, ohne das konzeptionelle Schema anpassen zu müssen. Logische Datenunabhängigkeit liegt vor, wenn das konzeptionelle Schema geändert werden kann, ohne das externe Schema anpassen zu müssen. Die in dieser Arbeit vorgestellten Konzepte beziehen sich primär auf das konzeptionelle Schema, wenngleich Schema-Versionierung alle drei Ebenen der ANSI-SPARC-Architektur betreffen kann.

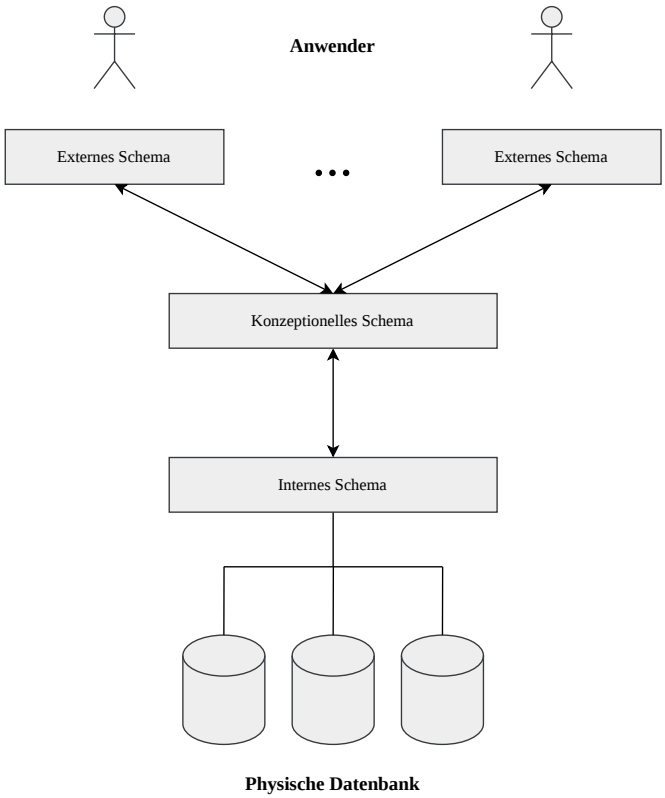



**Abbildung 1: ANSI-SPARC-Architektur**

Das Schema ist eine intensionale Beschreibung der Datenbank und enthält selbst keine Daten. Die *Datenbankinstanz* (im Folgenden kurz *Instanz*) hingegen repräsentiert den konkreten Zustand einer Datenbank zu einem bestimmten Zeitpunkt. Sie wird auch als

Extension bezeichnet und enthält die gespeicherten Daten gemäß den Strukturen und Integritätsbedingungen des Schemas. Jedes Einfügen, Löschen oder Aktualisieren von Daten führt zu einem neuen Zustand und damit zu einer neuen Instanz. Das Schema bleibt dabei konstant und ist zeitlich wesentlich stabiler als die Instanz. Erst wenn die Struktur der Datenbank modifiziert werden soll, muss das Schema angepasst werden. Dies kann beispielsweise eintreten, wenn sich die Anforderungen an die Applikation ändern, die mit der Datenbank interagiert. Die Kapitel 2.2 bis 2.4 gehen näher darauf ein, wie Änderungen an Schemata in einem DBMS unterstützt werden können.

„Dies ist ein direktes Zitat."[1, S. 212] Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. [2][3][4, S. 34]

1. First itemtext
2. Second itemtext
3. Last itemtext
4. First itemtext
5. Second itemtext

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris.

**Abbildung 2: Terminal**

```
fomtextemplatesvc_1  |   __ _ _ __ __ _ _ __ __ _
fomtextemplatesvc_1  |  / _` | '__/ _` | '__/ _` |
fomtextemplatesvc_1  | | (_| | | | (_| | | | (_| |
fomtextemplatesvc_1  |  \__,_|_|  \__,_|_|  \__,_|
fomtextemplatesvc_1  |
fomtextemplatesvc_1  | Processing 'elaborat.tex' (size: 10.0 kB, last modified:
fomtextemplatesvc_1  | 08/15/2021 17:30:02), please wait.
fomtextemplatesvc_1  |
fomtextemplatesvc_1  | (Clean) Cleaning feature ............................... SUCCESS
fomtextemplatesvc_1  | (Clean) Cleaning feature ............................... SUCCESS
fomtextemplatesvc_1  | (Clean) Cleaning feature ............................... SUCCESS
fomtextemplatesvc_1  | (XeLaTeX) XeLaTeX engine ............................... SUCCESS
fomtextemplatesvc_1  | (Nomencl) The Nomenclature software .................... SUCCESS
fomtextemplatesvc_1  | (Biber) The Biber reference management software ......... SUCCESS
fomtextemplatesvc_1  | (Biber) The Biber reference management software ......... SUCCESS
fomtextemplatesvc_1  | (XeLaTeX) XeLaTeX engine ............................... SUCCESS
fomtextemplatesvc_1  | (XeLaTeX) XeLaTeX engine ............................... SUCCESS
fomtextemplatesvc_1  | (Clean) Cleaning feature ............................... SUCCESS
fomtextemplatesvc_1  | (Clean) Cleaning feature ............................... SUCCESS
fomtextemplatesvc_1  | (Clean) Cleaning feature ............................... SUCCESS
fomtextemplatesvc_1  | (Clean) Cleaning feature ............................... SUCCESS
fomtextemplatesvc_1  | (Clean) Cleaning feature ............................... SUCCESS
fomtextemplatesvc_1  | (Clean) Cleaning feature ............................... SUCCESS
fomtextemplatesvc_1  | (Clean) Cleaning feature ............................... SUCCESS
fomtextemplatesvc_1  | (Clean) Cleaning feature ............................... SUCCESS
fomtextemplatesvc_1  | (Clean) Cleaning feature ............................... SUCCESS
fomtextemplatesvc_1  | (Clean) Cleaning feature ............................... SUCCESS
fomtextemplatesvc_1  | (Clean) Cleaning feature ............................... SUCCESS
fomtextemplatesvc_1  | (Clean) Cleaning feature ............................... SUCCESS
fomtextemplatesvc_1  |
fomtextemplatesvc_1  | Total: 55.03 seconds
fomtextemplatesvc_1  | Word Count:
fomtextemplatesvc_1  | 31+25+2 (11/2/0/0) File: /usr/fomtextemplate/deine_inhalte/Kapitel.tex
```

Quelle: [7, S. 200]

Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. [5, S. 415–426][6, S. 223]

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

### 2.1.1 Tabelle

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ull-

amcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue

a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

[4, S. 34][8]

- First itemtext
- Second itemtext
- Last itemtext
- First itemtext
- Second itemtext

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. (vgl. Tabelle **??**).[5, S. 511]

### 2.1.2 Quelltext

**Tabelle 1: Tabelle Eins**

| Spalte 1 | Spalte 2 | Spalte 3 | Spalte 4 | Spalte 5 |
|---|---|---|---|---|
| 1 | 2 | 3 | 4 | Lorem ipsum dolor sit amet |
| 1 | 2 | 3 | 4 | Lorem ipsum dolor sit amet |
| 1 | 2 | 3 | 4 | Lorem ipsum dolor sit amet |

Quelle: [7, S. 207]

```
apiVersion: v1
kind: Template
metadata:
  name: redis-template
  annotations:
  [...]
parameters:
 - description: Password used for Redis authentication
   from: '[A-Z0-9]{8}'
   generate: expression
   name: REDIS_PASSWORD
labels:
  redis: master
```

**Quelltext 1: Writing Templates**

Quelle: [9]

### 2.1.3 LaTeX-Befehle

Normale Schrift • **fette Schrift** • *kursive Schrift* • *schiefe Schrift* • unterstrichen • `Schreibmaschine` • Sans Serif • Roman

## 2.2 Schemamodifikation

Änderungen an der Definition des Schemas einer bestehenden Datenbank, werden auch als *Schemamodifikation* bezeichnet. Schemamodifikation umfasst das Hinzufügen, Mo-

difizieren und Entfernen von Elementen des Schemas. Bei der Schemamodifikation wird das bestehende Schema inklusive der zugehörigen Instanz verworfen und durch ein neues ‚leeres Schema ersetzt. Aspekte wie Datenerhalt und Schemakonsistenz werden dabei – im Gegensatz zur Schema-Evolution und Schema-Versionierung – nicht berücksichtigt. (siehe Seite 7 des Lietarture Reviews zur Schemaevolution für referenz)

Die Abbildungen 3 und 4 verdeutlichen den Ablauf einer Schemamodifikation in einer relationalen Datenbank. Ausgangspunkt ist die Schemaversion $\mathrm{SV}_1$ mit dem entsprechenden Eintrag im Systemkatalog des DBMS. $\mathrm{SV}_1$ besteht aus der Relation *EMPLOYEE* mit den Attributen *NAME*, *ADDRESS* und *CITY*. Die Instanz $I_1$ umfasst zwei Tupel (vgl. Abb. 3).

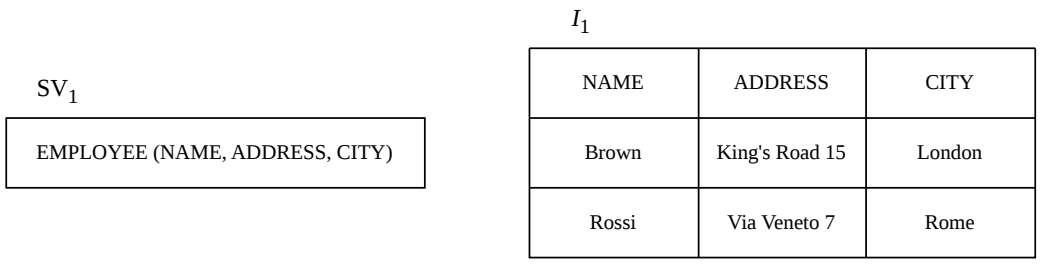



**Abbildung 3: Zustand vor der Schemamodifikation**

Das Schema wird mit Hilfe von DDL-Statements verändert (vgl. Quelltext 2). Zunächst wird das Attribut *ADDRESS* aus der Relation entfernt. Anschließend wird ein neues Attribut *PHONE* hinzugefügt.

```
ALTER TABLE EMPLOYEE
    DROP COLUMN ADDRESS;

ALTER TABLE EMPLOYEE
    ADD COLUMN PHONE CHAR(8);
```

**Quelltext 2: DDL-Statements**

Dadurch entsteht die neue Schemaversion $\mathrm{SV}_2$ mit leerer Instanz $I_2$ und angepasstem Eintrag im Systemkatalog (vgl. Abb. 4). Die ursprüngliche Schema-Version $\mathrm{SV}_1$ existiert nach der Modifikation nicht mehr. Auch die mit $\mathrm{SV}_1$ verbundene Instanz $I_1$ geht verloren.

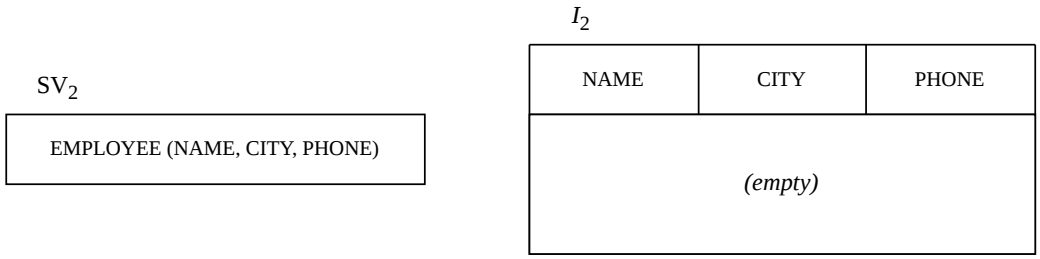



**Abbildung 4: Zustand nach der Schemamodifikation**

Das Beispiel zeigt damit die charakteristischen Eigenschaften einer klassischen Schemamodifikation: Änderungen werden direkt auf dem bestehenden Schema durchgeführt, ohne frühere Schemaversionen oder historische Instanzen beizubehalten. Anwendungen und Anfragen müssen anschließend zwingend mit der neuen Struktur kompatibel sein.

## 2.3 Schema-Evolution

*Schema-Evolution* bezeichnet die Änderung des Schemas einer bestehenden Datenbank, wobei die bereits gespeicherten Daten durch geeignete Migrationsoperationen an das neue Schema angepasst werden. Sie grenzt sich von der Schemamodifikation dadurch ab, dass die bestehenden Daten erhalten bleiben und in das neue Schema überführt werden, anstatt sie zu verwerfen. Dennoch kann es im Rahmen der Schema-Evolution, beispielsweise bei der Entfernung von Attributen oder der Änderung von Datentypen, zu einem teilweisen Informationsverlust kommen. Auch die Kompatibilität des neuen Schemas mit Anwendungen, die auf Basis des alten Schemas entwickelt wurden, ist durch Schema-Evolution nicht garantiert.

Abbildung 5 zeigt das aus den DDL-Statements in Quelltext 2 resultierende Schema bei einem DBMS, das Schema-Evolution unterstützt. Die ursprüngliche Schemaversion $\mathrm{SV}_1$ wird verworfen und vollständig durch die neue Schemaversion $\mathrm{SV}_2$ ersetzt. Ebenso wird die bestehende Instanz $I_1$ vollständig durch die neue Instanz $I_2$ ersetzt. Die Daten der Attribute *NAME* und *CITY* werden auf Basis von $I_1$ in die Instanz $I_2$ der neuen Schemaversion $\mathrm{SV}_2$ übernommen. Die Informationen des Attributes *ADDRESS* gehen hingegen verloren, da dieses kein Bestandteil der Schemaversion $\mathrm{SV}_2$ ist. Für das neu eingeführte

Attribut *PHONE* liegen zunächst keine Werte vor, sodass die entsprechenden Tupel mit NULL-Werten belegt werden.

$SV_2$

| EMPLOYEE (NAME, CITY, PHONE) |
|---|

$I_2$

| NAME | CITY | PHONE |
|---|---|---|
| Brown | London | *Null* |
| Rossi | Rome | *Null* |

**Abbildung 5: Zustand nach der Schema-Evolution**

## 2.4 Schema-Versionierung

*Schema-Versionierung* bezeichnet die Fähigkeit eines Datenbanksystems, verschiedene Zustände eines Schemas über die Zeit hinweg zu verwalten und zugänglich zu machen. Dabei werden Änderungen am Schema historisch gespeichert, sodass frühere und aktuelle Versionen weiterhin genutzt werden können. Im Unterschied zur Schema-Evolution wird das bestehende Schema nicht ersetzt, sondern durch eine zusätzliche Schemaversion ergänzt. Folglich können mehrere Schemaversionen parallel im DBMS zur Laufzeit existieren. Ein wesentliches Merkmal ist zudem die Möglichkeit, Daten sowohl über aktuelle als auch historische Schemata abzurufen. Das ermöglicht Anwendungen auf jene Schemaversion zuzugreifen, für die sie entwickelt wurden, wodurch ihre Funktionsfähigkeit erhalten bleibt. Neue Daten werden dabei in der Regel nach der aktuellen Schemaversion gespeichert. Ein zentrales Ziel der Schema-Versionierung ist es, Informationsverlust zu vermeiden und den Betrieb bestehender Anwendungen weiterhin zu gewährleisten.

Schema-Versionierung ist komplexer als die reine Schema-Evolution, da sämtliche Schemaversionen innerhalb eines DBMS gemeinsam mit den zugehörigen Instanzen und abhängigen Komponenten koexistieren und konsistent betrieben werden müssen.

Abbildung 6 zeigt das aus den DDL-Statements in Quelltext 2 resultierende Schema bei einem DBMS, das Schema-Versionierung unterstützt. Im Gegensatz zur Schema-Evolution

koexistiert die neue Schemaversion $\mathrm{SV}_2$ mit der ursprünglichen Schemaversion $\mathrm{SV}_1$ innerhalb des DBMS. Dies wird unter anderem durch die beiden Einträge im Systemkatalog deutlich. Die Instanz $I_1$ der Schemaversion $\mathrm{SV}_1$ bleibt vollständig erhalten. Für $\mathrm{SV}_2$ werden die Daten analog zur Schema-Evolution an die neue Schemastruktur angepasst, sodass die Instanz $I_2$ entsteht. Dadurch können Anwendungen, die auf Basis von $\mathrm{SV}_1$ oder $\mathrm{SV}_2$ entwickelt wurden, weiterhin ohne Anpassungen betrieben werden.

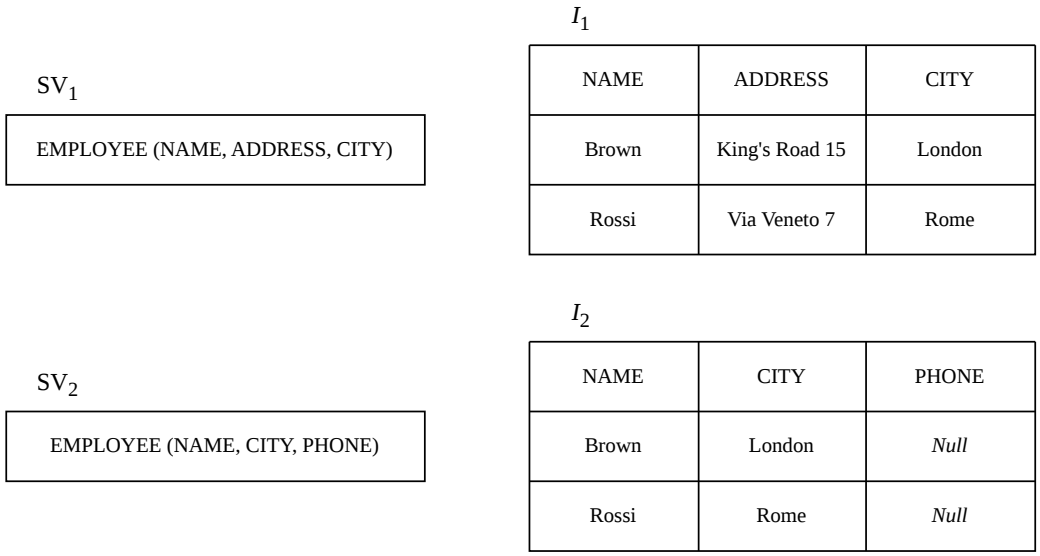



**Abbildung 6: Zustand nach der Schema-Versionierung**

# 3 Methoden der Schema-Versionierung

Im folgenden Kapitel werden die grundlegenden Konzepte der Schema-Versionierung vorgestellt, die in zahlreichen Forschungsarbeiten als Basis unterschiedlicher Lösungsansätze dienen. Ziel des Kapitels ist die Vermittlung eines allgemeinen Verständnisses der zugrunde liegenden Prinzipien und Mechanismen der Schema-Versionierung, ohne dabei auf die detaillierte Implementierung einzelner Ansätze einzugehen. Da die konkrete Ausgestaltung einer Schema-Versionierung maßgeblich von den jeweiligen Anwendungsanforderungen abhängt, ist eine isolierte Betrachtung einzelner Lösungen nur eingeschränkt aussagekräftig. Aus diesem Grund werden spezifische Forschungsergebnisse und prototypische Implementierungen im Rahmen dieser Arbeit nicht vertieft behandelt.

## 3.1 Dimensionen der Schema-Versionierung

### 3.1.1 Partielle und vollständige Schema-Versionierung

Im Hinblick auf die Lese- und Schreib-Operationen, welche auf die Daten eines Schemas ausgeführt werden können, wird in der Literatur zwischen *partieller Schema-Versioning* und *vollständiger Schema-Versioning* differenziert.

Die *partielle Schema-Versioning* ermöglicht das Abrufen von Daten über alle vergangenen und zukünftigen Schemaversionen hinweg. Änderungen an den Daten können jedoch nur über eine einzige – in der Regel die aktuelle – Schemaversion vorgenommen werden. Daher gewährleistet die partielle Schema-Versionierung nicht, dass bestehende Anwendungen nach einer Schemaänderung weiterhin fehlerfrei funktionieren, sofern diese Anwendungen Daten modifizieren. Abbildung 7 verdeutlicht, dass die Instanzen $I_1$ und $I_2$ sowohl über $\mathrm{SV}_1$ als auch über $\mathrm{SV}_2$ abrufbar sind. Schreib-Operationen können jedoch nur über $\mathrm{SV}_2$ auf $I_2$ ausgeführt werden. Anwendungen, die Schreib-Operationen über $\mathrm{SV}_1$ ausführen, funktionieren daher nach der Einführung von $\mathrm{SV}_2$ nicht mehr korrekt. Ab diesem Zeitpunkt sind Schreib-Operationen nur noch über $\mathrm{SV}_2$ möglich.

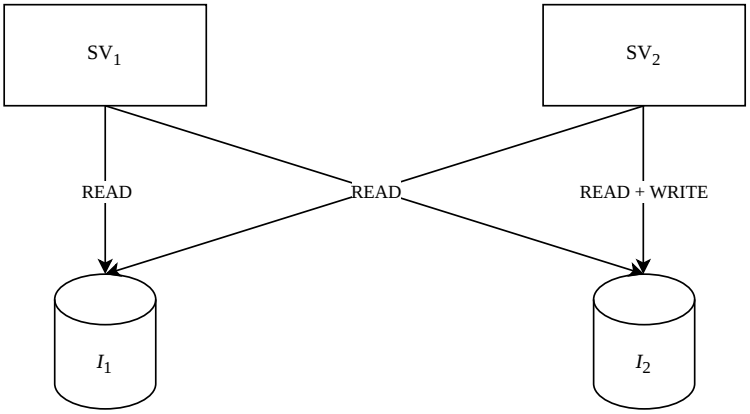



**Abbildung 7: Partielle Schema-Versionierung**

Die *vollständige Schema-Versioning* hingegen ermöglicht sowohl das Lesen als auch das Schreiben von Daten über alle vergangenen und zukünftigen Schemaversionen hinweg. Dadurch bleiben bereits kompilierte Anwendungen auch nach beliebigen Schemaänderungen weiterhin voll funktionsfähig. Das Lesen und Schreiben ist hierbei über $\mathrm{SV}_1$ und $\mathrm{SV}_2$ auf die Instanzen $I_1$ und $I_2$ möglich (vgl. Abb. 8).

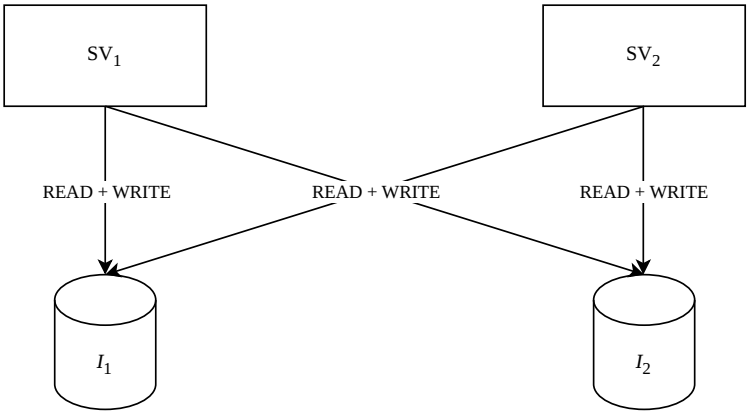



**Abbildung 8: Vollständige Schema-Versionierung**

Die Kompatibilität zwischen Instanzen und Schemaversionen ist ein zentrales Problem der Schema-Versionierung. In diesem Kontext unterscheidet die Literatur zwischen *Rückwärtskompatibilität* und *Vorwärtskompatibilität*. Rückwärtskompatibilität bedeutet, dass Daten, die unter einer früheren Schemaversion erstellt wurden, von einer Anwendung verarbeitet werden können, die für eine aktuellere Schemaversion entwickelt wurde (beispielsweise der Zugriff auf $I_1$ über $\mathrm{SV}_2$). Vorwärtskompatibilität bezeichnet entsprechend die Fähigkeit, dass Daten, die unter einer neueren Schemaversion erstellt wurden, von einer Anwendung verarbeitet werden können, die für eine ältere Schemaversion konzipiert ist (beispielsweise der Zugriff auf $I_2$ über $\mathrm{SV}_1$). Zusammenfassend betreffen Rückwärts- und Vorwärtskompatibilität die Möglichkeit, auf Daten über eine andere Schemaversion zuzugreifen als jene, mit der die Daten ursprünglich erzeugt wurden.

In der Forschung wird diskutiert, ob Schemaänderungen vollständig verlustfrei und kompatibel umgesetzt werden können. Da Änderungen an Schemata häufig die Menge gültiger Daten verändern, ist vollständige Vorwärts- und Rückwärtskompatibilität theoretisch oft nicht realisierbar. Aus diesem Grund unterstützen viele Systeme lediglich die partielle Schema-Versionierung.

### 3.1.2 Temporale Schema-Versionierung

Neben nicht-temporalen Konzepten, können Schemata auch entlang zeitlicher Dimensionen versioniert werden. De Castro et al. entwickelten – aufbauend auf einem Vorschlag

von John F. Roddick – drei Ansätze der temporalen Schema-Versionierung. Dabei unterscheiden sie zwischen *transaktionszeitbasierter (transaction-time)*, *gültigkeitszeitbasierter (valid-time)* und *bitemporaler (bitemporal-time)* Schema-Versionierung.

**Transaktionszeit**

Bei der *transaktionszeitbasierten Schema-Versionierung* versieht das DBMS jede Schemaversion mit einem Transaktionsbeginn und einem Transaktionsende als Zeitstempel. Die Kataloginformationen werden dabei analog zu Transaktionszeittabellen verwaltet. Änderungen können ausschließlich an der aktuell gültigen Schemaversion vorgenommen werden. Bereits vergangene Versionen bleiben unveränderlich, während zukünftige Versionen im Rahmen dieses Ansatzes nicht berücksichtigt werden. Dieses Verfahren eignet sich insbesondere für Anwendungsszenarien, in denen Schemaänderungen unmittelbar nach ihrer Durchführung wirksam werden sollen.

**Gültigkeitszeit**

In der *gültigkeitszeitbasierten Schema-Versionierung* wird jede Version eines Schemas durch einen Beginn sowie ein Ende ihrer zeitlichen Gültigkeit beschrieben. Hierfür verwaltet das DBMS die Kataloginformationen in Form von Gültigkeitszeittabellen. Da sich dieser Ansatz an der Gültigkeitszeit orientiert, lassen sich nicht nur unmittelbar wirksame Änderungen am Schema abbilden, sondern auch rückwirkende sowie zukünftige Anpassungen. Rückwirkende Änderungen entstehen, wenn eine reale Veränderung bereits eingetreten ist und erst nachträglich im Schema nachvollzogen wird. Zukünftige Änderungen hingegen werden bereits vor ihrem tatsächlichen Wirksamwerden definiert und mit einer entsprechenden zukünftigen Gültigkeitsperiode versehen. Dadurch können frühere, aktuelle und geplante Versionen eines Schemas gezielt angepasst werden. Die Versionierung basierend auf Gültigkeitszeit eignet sich insbesondere für Anwendungsbereiche, in denen Schemaänderungen mit rückwirkender oder vorausplanender Wirkung erforderlich sind. Rückwirkende Anpassungen können beispielsweise durch gesetzliche Vorgaben notwendig werden, die strukturelle Änderungen administrativer Datenbestände verlangen – wie etwa eine Anpassung der Ziffernlänge der Sozialversicherungsnummer. Zukünftige Schemaänderungen bieten dagegen die Möglichkeit, geplante Anpassungen bereits im Vorfeld

zu modellieren, zu testen und hinsichtlich ihrer Auswirkungen zu bewerten, bevor sie produktiv eingesetzt werden.

**Bitemporale-Zeit**

Bei der *bitemporalen Schema-Versionierung* wird jede Version eines Schemas sowohl mit Transaktionszeit- als auch mit Gültigkeitszeitstempeln versehen, sodass die Datenbankkataloge bitemporal organisiert sind. Die Nutzung beider Zeitdimensionen ist insbesondere für Anwendungen relevant, die Schemaänderungen nicht nur zeitnah, sondern auch rückwirkend oder im Voraus ermöglichen müssen und bei denen diese Modifikationen lückenlos nachvollziehbar bleiben sollen. Dadurch wird im Systemkatalog dokumentiert, ob eine Änderung retroaktiv oder proaktiv erfolgt ist. Im Gegensatz dazu erlaubt eine rein gültigkeitszeitbasierte Schema-Versionierung nach der Ausführung keine eindeutige Rekonstruktion, ob eine Schemaänderung rückwirkend oder vorausschauend eingetragen wurde. Durch die zusätzliche Erfassung der Transaktionszeit bleibt hingegen exakt dokumentiert, zu welchem Zeitpunkt die Änderung tatsächlich in das System übernommen wurde.

Es folgt eine beispielhafte Illustration der Ansätze. Die Beispiele basieren auf der Arbeit von De Castro et al.

Gegeben seien folgende Datenbank-Operationen:

O1 – Schemaänderung – On-time-Transaktion – committed am 1983/1/1 – wirksam von 1983/1/1 bis 1985/12/31: Erstellung der Tabelle EMPLOYEE mit den Spalten NAME, ADDRESS, CITY;

O2 – Datenänderung – On-time-Transaktion – committed am 1984/1/1: Einfügen des Tupels „(Blanc, BD St Michel 87, Paris)“ in die Tabelle EMPLOYEE;

O3 – Schemaänderung – rückwirkende (retroaktive) Transaktion – committed am 1985/1/1 – wirksam von 1983/1/1 bis 1985/12/31: Entfernung der Spalte ADDRESS aus der Tabelle EMPLOYEE (die Transaktion modifiziert die vorherige Schemaversion, ohne ihre Gültigkeit zu beeinträchtigen);

O4 – Datenänderung – On-time-Transaktion – committed am 1986/1/1: Aktualisierung des Attributs CITY (gesetzt auf Bruxelles) in Blancs Daten, die in der Tabelle EMPLOYEE gespeichert sind;

O5 – Schemaänderung – proaktive Transaktion – committed am 1987/1/1 – wirksam ab 1988/1/1: Hinzufügen der Spalte PHONE zur Tabelle EMPLOYEE.

Die nachfolgenden Abbildungen 9-11 beschreiben den Zustand des DBMS nach Ausführung der Operationen O1-O5. Der Einfachheit halber wird eine zeitliche Granularität von einem Tag angenommen. Zudem wird auf eine Angabe der erforderlichen SQL-Statements verzichtet.

Transaction-Time Schema Versioning: Die Operation O1 initialisiert das Datenbankschema, bestehend aus der Relation EMPLOYEE mit den Attributen NAME, ADDRESS und CITY. Nach der transaktionszeitbasierten Schema-Versionierung, dürfen Anpassungen nur die aktuelle Schema-Version betreffen. Schemaänderungen werden daher ausschließlich als on-time Transaktionen ausgeführt. Sie werden also mit dem Zeitpunkt ihrer Definition auf das aktuelle Schema wirksam. Damit ist eine Schema-Version immer so lange aktuell, bis eine neue Schemaänderung definiert und damit eine neue Schema-Version erzeugt wird. Der Status im Systemkatalog des DBMS nach O1 wäre daher: SV1 (1/1/83 - infinity), bis zu dem Zeitpunkt einer neuen Schemaänderung. O2 und O4 betreffen die Datenbankinstanz. Sie verändern das Schema nicht und haben damit keinen Einfluss auf den Systemkatalog. O3 und O5 sind retro- bzw. proaktive Schemaänderungen. Da die transaktionszeitbasierte Versionierung diese Art der Anpassung nicht unterstützt, werden die Schemaänderungen ebenfalls mit dem Zeitpunkt ihrer Definition wirksam, wie der Abbildung 9 zu entnehmen ist. O3 erzeugt die Schema-Version SV2 und O5 die Schema-Version SV3.

Valid-Time Schema Versioning: Im Gegensatz zum Transaction-time schema versioning, kann nach diesem Ansatz der Gültigkeitszeitraum der Schemaänderung definiert werden. Die Anpassungen betreffen dann jenes Schema, das für diesen Gültigkeitszeitraum definiert ist. Dadurch sind Anpassungen an vergangenen oder zukünftigen Schema-Versionen möglich. Da die retroaktive Schemaänderungen O3 für den gleichen Gültigkeitszeitraum definiert ist, wie die Schema-Version SV1, ersetzt die aus O3 resultierende Version SV2 die alte Version SV1 vollständig. SV1 wird gelöscht und ist in der Datenbank und dem

DBMS somit nicht mehr existent. Die proaktive Schemaänderung O5 ist für einen Gültigkeitszeitraum in der Zukunft definiert. O5 wird somit erst am 1/1/88 ausgeführt, wodurch SV3 entsteht. In der Arbeit von De Castro et al. wird nicht näher erläutert, welche Version im Zeitraum von 86-88 greift.

Bitemporal Schema-Versioning: Die Verwaltung der Schema-Versionen entlang der Transaktionszeit und der Gültigkeitszeit ermöglicht es, historische Status der Datenbank zu bewahren. Ein Rückgang auf ältere Schema-Versionen ist somit möglich. Schemata die für den gleichen Gültigkeitszeitraum definiert sind, müssen nicht mehr gelöscht werden, da die Transaktionszeit belegt, welche Schema-Version aktuell ist. Die aus O3 resultierende Version SV2 würde die Version SV1 zwar hinsichtlich der Aktualität ablösen. SV1 wird nach diesem Ansatz jedoch nicht gelöscht, sondern archiviert und ist weiterhin abrufbar, sofern ein Rückgang von SV2 auf SV1 erforderlich ist. Sowohl die intensionalen als auch die extensionalen Daten bleiben dabei erhalten.

### 3.1.3 Speicherkonzept

Die Koordination zwischen Schema-Version und Datenbankinstanz betreffend, unterscheiden De Castro et al. zwei grundlegende Speicher-Strategien: den Single-Pool- und den Multi-Pool-Ansatz. Der Begriff Data-Pool meint in diesem Kontext einen Speicherort für extensionale Daten. Beide Ansätze sind auf logischer Ebene zu betrachten. Details der physischen Speicherung wurden von den Autoren absichtlich abstrahiert.

Beim Multi-Pool-Ansatz werden die Daten jeder Version einer Relation in einem separaten Speicher gehalten. Die Daten der unterschiedliche Versionen werden also unabhängig voneinander gespeichert. Ausgehend von dem Beispiel aus Kapitel 2.2: Werden die Instanzen I1 und I2 jeweils in eigenen Tabellen abgelegt, entspricht dies einer Speicherung nach dem Multi-Pool-Verfahren (Vgl. Abb. 12).

Der Single-Pool-Ansatz verfolgt dagegen das Ziel, alle Daten einer Relation zentral in einer gemeinsamen Struktur zu verwalten. Dadurch lassen sich redundante Daten vermeiden, insbesondere wenn mehrere Schemaversionen dieselben Werte verwenden. Für den Single-Pool Ansatz wird ein übergeordnetes Schema erzeugt, das sämtliche Attribute aller vorhandenen Schemaversionen der Relation umfasst. Dieses sogenannte „Completed

Schema“ dient als gemeinsame Grundlage für die Speicherung der Daten. Die einzelnen Versionen des Schemas werden anschließend nicht mehr direkt als eigene Tabellen geführt, sondern als Views auf diese gemeinsame Datenbasis definiert. Jede View stellt dabei nur die Attribute bereit, die für die jeweilige Schemaversion relevant sind. Abbildung 13 zeigt eine Single-Pool Implementierung des Beispieles aus Kapitel 2.2.. Die Instanzen von SV1 und SV2 werden dabei in einer einzigen Tabelle gehalten.

Den folgendne Part evtl ganz weglassen und nur auf Vor Nachteile verweisen:

Ein wesentlicher Unterschied zwischen beiden Ansätzen zeigt sich beim Aufwand zur Unterstützung von Vorwärts- und Rückwärtskompatibilität zwischen mehreren Versionen. Existieren insgesamt (n) Schemaversionen, so werden beim Single-Pool-Ansatz lediglich (n) Views benötigt, um Daten zwischen verschiedenen Versionen zugänglich zu machen und damit Vorwärts- sowie Rückwärtskompatibilität zu unterstützen. Beim Multi-Pool-Ansatz steigt der Aufwand deutlich an, da hierfür insgesamt (n(n-1)) Views beziehungsweise Konvertierungsfunktionen erforderlich sein können. Auch hinsichtlich der Änderbarkeit der Daten ergeben sich Unterschiede. Für die Unterstützung eines partiellen Schema-Versionierungsansatzes muss beim Single-Pool-Modell das aktuelle Schema als aktualisierbare View auf dem Completed Schema definiert sein. Soll dagegen vollständige Schema-Versionierung ermöglicht werden, müssen sämtliche Views aktualisierbar sein, was wesentlich schwieriger umzusetzen ist.

De Castro et al. und Brahmia et al. haben in ihren Arbeiten weitere Vor- und Nachteile beider Ansätze beluechtet auf die an dieser Stelle zwar verwiesen aber im Rahmen dieser Arbeit nicht weiter eingegangen wird. (Referenz geben wo das in den Arbeiten thematisiert wird).

### 3.1.4 Interaktion zwischen Instanz und Schema

In den bislang betrachteten Beispielen wurden ausschließlich klassische Snapshot-Daten betrachtet, da diese ausreichten, um die zuvor vorgestellten Ansätze zur Schema-Versionierung zu erläutern. Zusätzliche Herausforderungen entstehen jedoch, wenn Schema-Versionierung in Datenbanken eingesetzt wird, die temporale Daten speichern. In diesem Fall muss das Zusammenspiel zwischen der Versionierung der Daten selbst (extensionale Versionierung)

und der Versionierung des Schemas (intensionale Versionierung) berücksichtigt werden. Aus dieser Wechselwirkung ergibt sich eine weitere Gestaltungsoption für das Datenmanagement. Sie wird relevant, wenn sowohl Daten als auch Schemata entlang derselben Zeitdimension versioniert werden. Die Autoren unterscheiden dabei zwei grundlegende Verwaltungsansätze: • Synchrones Management: Temporale Daten werden ausschließlich über jene Schemaversion verarbeitet, deren zeitlicher Gültigkeitsbereich mit dem der Daten übereinstimmt. Das betrifft sowohl das Speichern als auch das Abrufen und Aktualisieren der Daten. • Asynchrones Management: Temporale Daten können unabhängig von ihrer zeitlichen Einordnung über beliebige Schemaversionen abgefragt oder verändert werden. Die zeitliche Gültigkeit von Daten und Schema muss hierbei also nicht übereinstimmen. Dieses Modell ist vor allem notwendig, um Rückwärts- und Vorwärtskompatibilität zu ermöglichen sowie partielle oder vollständige Schema-Versionierung zu unterstützen, wenn sowohl Daten als auch Schemata entlang derselben zeitlichen Dimension versioniert werden.

In diesem Kontext erwähnen die Autoren eine wichtige Einschränkung. Werden Schema und Daten über die Transaktionszeit verwaltet, ist nur synchrones Management möglich, weil diese Zeitdimension den tatsächlichen historischen Zustand der Datenbank beschreibt. Wenn eine Datenbank auf einen bestimmten Zeitpunkt zurückgesetzt wird, muss sowohl das Schema als auch die gespeicherten Daten genau zu diesem Zeitpunkt passen. Würde man Schema- und Datenversionen asynchron verwalten, könnten unterschiedliche zeitliche Zustände miteinander kombiniert werden — etwa ein Schema aus dem Jahr 1995 mit Daten aus dem Jahr 1996. Dadurch entstünde ein inkonsistenter Datenbankzustand, der historisch nie tatsächlich existiert hat. Genau das widerspricht jedoch der Bedeutung der Transaktionszeit, deren Zweck darin besteht, vergangene Zustände der Datenbank korrekt und vollständig rekonstruieren zu können. Deshalb müssen bei der Transaction Time Schema und Daten immer gemeinsam und zeitlich abgestimmt versioniert sowie abgefragt werden. Nur so bleibt gewährleistet, dass jeder rekonstruierte Zustand konsistent ist. Für die Valid Time gilt diese Einschränkung dagegen nicht, da sie nicht den historischen Zustand der Datenbank selbst beschreibt, sondern die fachliche Gültigkeit von Informationen in der realen Welt.

Das nachfolgende Beispiel soll die Funktionweisen beider Ansätze verdeutlichen. Der

Ausgangspunkt ist die Relation EMPLOYEE mit den Attributen NAME und SALARY. Sowohl Schema als auch Daten werden über die Gültigkeitszeit-Dimension versioniert. Die zwei Tupel zeigen die Gehaltshistorie des Mitarbeiters ”Brown”(vgl. Abb. 14).

Anschließend wird eine Schemaänderung angenommen, bei der der Relation EMPLOYEE das zusätzliche Attribut DEPT hinzugefügt wird. Diese Änderung soll ab dem 1. Januar 1994 gültig sein. Darüber hinaus wird angenommen, dass dieselbe Transaktion mithilfe der neuen Schemaversion die Information speichert, dass der Mitarbeiter Brown schon immer in der Sales-Abteilung gearbeitet hat. Abbildung 15 veranschaulicht den Zustand des DBMS nach der Schemaänderung im Rahmen des synchronen Managements. Es zeigt sich, dass die Abteilungsinformation für den Zeitraum 1992–1993 nicht gespeichert werden konnte, da die entsprechenden Daten von Brown in diesem Zeitraum noch mit der alten Schemaversion verknüpft sind, die das Attribut DEPT nicht enthält. Darüber hinaus müssen spezielle Anwendungen entwickelt werden, die beide Schemaversionen berücksichtigen, um selbst die vollständige Historie der älteren Daten – also die Daten ohne das Attribut DEPT – von 1990 bis heute rekonstruieren zu können. Der Grund dafür liegt darin, dass die Historie der Daten im synchronen Management analog zu den Schemaversionen aufgeteilt wird.

Abbildung 16 zeigt den Zustand des DBMS unter Anwendung von asynchronem Management: In diesem Fall bleibt die vollständige Historie der Mitarbeiterdaten in jeder Schemaversion erhalten. Bestehende Anwendungen können weiterhin mit den älteren Daten arbeiten und liefern dabei dieselben Ergebnisse wie vor der Schemaänderung. Gleichzeitig können neue Anwendungen entwickelt werden, die auf den erweiterten Datenbestand zugreifen und zusätzlich die Informationen des Attributs DEPT nutzen.

An dieser Stelle sei noch erwähnt, dass die konkreten Auswirkungen von Schema- und Datenänderungen im Rahmen des asynchronen Managements zusätzlich davon abhängen, ob Tabellen mithilfe eines Single-Pool- oder eines Multi-Pool-Ansatzes implementiert werden.

Dabei wird die vollständige Schema-Versionierung entlang der Transaction Time aus Konsistenzgründen eingeschränkt, während andere Ausprägungen weiterhin als mögliche Entwurfsentscheidungen betrachtet werden mit async oder sync management!!

Abschließend sei darauf hingewiesen, dass viele wissenschaftliche Lösungsansätze auf einer Kombination der zuvor vorgestellten Dimensionen und Konzepte basieren. Aufgrund verschiedener technischer und konzeptioneller Restriktionen sind jedoch nicht alle Kombinationen dieser Konzepte miteinander kompatibel. So stellt bei Daten und Schemata, die entlang derselben Zeitdimension versioniert werden, die Gültigkeitszeit in Verbindung mit asynchronem Management die einzige Kombination dar, die mit einer partiellen oder vollständigen Schema-Versionierung vereinbar ist.

Ein Beispiel für einen Lösungsansatz, der eine vollständige Schema-Versionierung unterstützt, ist der Prototyp VERSIOS. Dieser basiert auf einer Versionierung entlang der Gültigkeits- beziehungsweise bitemporalen Zeit, verwendet das Multi-Pool-Verfahren und verwaltet die Interaktion zwischen Schema- und Instanzversionen durch asynchrones Management.

## 3.2 DBMS Support und Tooling

Auch wenn die Schema-Versionierung theoretisch die einzige Methode ist, mit der sich Änderungen an einer sich entwickelnden Datenbankstruktur sauber nachverfolgen und auch frühere Zustände wiederherstellen lassen, wird dieses Konzept von heutigen relationalen DBMS nicht nativ unterstützt. Auch der SQL-Standard definiert kein Konzept für Schema-Versionierung.

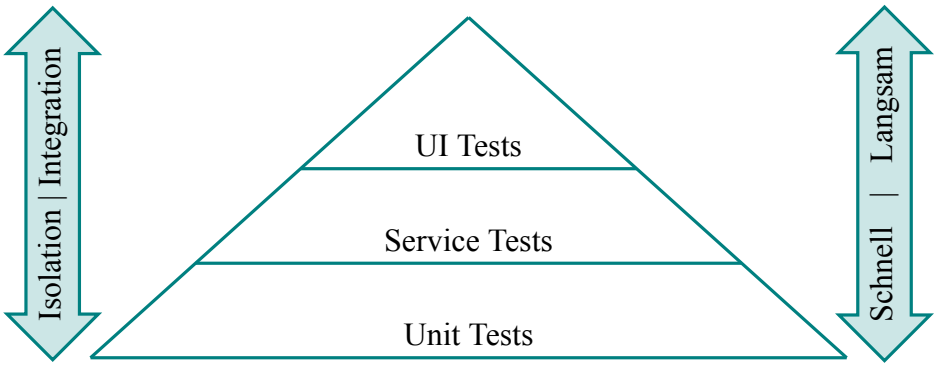
Viele kommerzielle Datenbanksysteme wie Microsoft SQL Server, IBM DB2, Oracle oder MySQL stellen über integrierte Funktionen oder die Unterstützung externer Werkzeuge eine Versionsverwaltung für die Entwicklung von Schemata zur Verfügung. Dadurch können Datenbankentwickler parallel an einem noch nicht produktiven Datenbankschema arbeiten und verschiedene Varianten sowie aufeinanderfolgende Versionen erstellen. Diese Versionierungsmechanismen helfen beispielsweise dabei, Änderungen verschiedener Entwickler zusammenzuführen, fehlerhafte Anpassungen gezielt zurückzunehmen, Unterschiede zwischen Entwicklungsständen oder Schema-Varianten nachzuvollziehen und Skripte zu erzeugen, die ein Schema in eine andere Version überführen. Eine solche Überführung kann dabei auch vorhandene Daten betreffen und stellt somit eine teilweise und oft verlustbehaftete Unterstützung der Schema-Evolution vor der Produktivsetzung dar.

Tools wie Flyway oder Liquibase unterstützen diese Funktion. Obwohl diese Tools häufig der „Schema-Versionierung“ zugeordnet werden, handelt es sich dabei nicht um Schema-Versionierung im Sinne der Endnutzer: In produktiven Umgebungen können bestehende Daten nicht gleichzeitig über verschiedene Schema-Versionen hinweg geteilt oder unterschiedlich angezeigt und bearbeitet werden.

Eine Lösung die dem Gedanken der Schema-Versionierung nahe kommt ist der Oracle Workspace Manager.

Oracle Workspace Manager stellt eine datenbankinterne Versionierungsmechanik bereit, die auf Zeilenebene mehrere konsistente Datenzustände innerhalb sogenannter Workspaces ermöglicht. Im Gegensatz zu klassischen Migrationsansätzen erlaubt das System damit eine parallele Existenz unterschiedlicher Datenversionen innerhalb derselben Datenbankinstanz. Dadurch ist OWM dem Konzept der Schema- bzw. Datenversionierung konzeptionell sehr nahe, da es nicht nur eine zeitliche Evolution, sondern auch gleichzeitige, isolierte Zustände unterstützt. Allerdings handelt es sich nicht um eine vollständige Schema-Versionierung, da insbesondere die strukturelle Unabhängigkeit von Tabellen und Schemata nicht gegeben ist und Versionierung primär auf Daten- und nicht auf Strukturebene erfolgt.

**Abbildung 9: Testpyramide**




Quelle: Eigendarstellung angelehnt an [10, S. 311–317]

## 3.3 Ausblick

Die standardisierte SQL-DDL (Data Definition Language), die für die Definition und Modifikation relationaler Datenbankschemata eingesetzt wird, weist lediglich eingeschränkte Möglichkeiten zur Unterstützung von Schemaänderungen auf und bietet keine native Funktionalität für Schema-Evolution oder Schema-Versionierung. Aus diesem Grund erscheint eine Erweiterung der SQL-Spezifikation sinnvoll, die explizit Konzepte der Schema-Evolution und Versionierung auf Benutzerebene unterstützt. Eine solche Weiterentwicklung könnte dazu beitragen, Änderungen am Datenbankschema nicht nur technisch, sondern auch konzeptionell transparenter und benutzerfreundlicher abzubilden. Im Hinblick auf zukünftige Forschungsarbeiten eröffnet sich hierbei ein breites Spektrum an Fragestellungen. Insbesondere die Entwicklung standardisierter Sprachkonstrukte für Schema-Versionierung, die Integration von Migrationsmechanismen direkt in die Datenbanksprache sowie die konsistente Unterstützung historisierter Datenabfragen über mehrere Schemazustände hinweg stellen vielversprechende Forschungsrichtungen dar. Darüber hinaus wäre zu untersuchen, wie sich solche Erweiterungen in bestehende Datenbankmanagementsysteme integrieren lassen, ohne deren Komplexität oder Performance wesentlich zu beeinträchtigen.

# Literaturverzeichnis

[1] P. Mertens, D. Barbian und S. Baier, *Digitalisierung und Industrie 4.0 - eine Relativierung*, eng. Wiesbaden, Germany: Springer Vieweg, 2017, OCLC: 1028237558, ISBN: 9783658196325 9783658196318.

[4] M.-C. Rische und H. Vöpel, „Schwerpunkt Kreative Zerstörung 4.0 - Die Neuvermessung der Welt - Grundprinzipien und Konsequenzen der Digitalökonomie," Wirtschaftspolitische Blätter, Nr. 2, 2016.

[5] A. S. Tanenbaum und H. Bos, *Moderne Betriebssysteme*, 4. Aufl., Ser. Pearson Studium - IT. Hallbergmoos: Pearson, 2016, ISBN: 978-3-86894-270-5.

[6] P. Mandl, *Internet Internals: Vermittlungsschicht, Aufbau und Protokolle*, ger, Ser. Lehrbuch. Wiesbaden: Springer Fachmedien Wiesbaden GmbH, 2019, OCLC: 1056585008, ISBN: 9783658235369 9783658235352.

[7] M. Mustermann, *Eine beispielhafte Quelle*, 9. Aufl. München: Beispiel Verlag, 2020.

[10] M. Cohn, *Succeeding with agile: software development using Scrum*, English. Upper Saddle River, N.J.: Addison-Wesley, 2010, OCLC: 489135509, ISBN: 9780321660510 9786612430565 9781282430563 9780321660565. Adresse: http://search.ebscohost.com/login.aspx?direct=true&scope=site&db=nlebk&db=nlabk&AN=1599331.

# Internetquellen

[2] Microsoft. (2019). „Übersicht über die Datenbindung in WPF," Adresse: https://docs.microsoft.com/de-de/dotnet/desktop-wpf/data/data-binding-overview (besucht am 24. 05. 2020).

[3] M. Hausenblas und N. Bijnens. (2017). „Lambda Architecture," Adresse: http://lambda-architecture.net/ (besucht am 14. 08. 2021).

[8] Z. Dehghani. (2019). „How to Move Beyond a Monolithic Data Lake to a Distributed Data Mesh," Adresse: https://martinfowler.com/articles/data-monolith-to-mesh.html (besucht am 22. 08. 2021).

[9] Red Hat. (2021). „Templates | Developer Guide | OpenShift Container Platform 3.7,“ Adresse: https://docs.openshift.com/container-platform/3.7/dev_guide/templates.html (besucht am 14. 12. 2021).

# Eigenständigkeitserklärung

Hiermit versichere ich, dass ich die angemeldete Prüfungsleistung in allen Teilen eigenständig ohne Hilfe von Dritten anfertigen und keine anderen als die in der Prüfungsleistung angegebenen Quellen und zugelassenen Hilfsmittel verwenden werde. Sämtliche wörtlichen und sinngemäßen Übernahmen inklusive KI-generierter Inhalte werde ich kenntlich machen.

Diese Prüfungsleistung hat zum Zeitpunkt der Abgabe weder in gleicher noch in ähnlicher Form, auch nicht auszugsweise, bereits einer Prüfungsbehörde zur Prüfung vorgelegen; hiervon ausgenommen sind Prüfungsleistungen, für die in der Modulbeschreibung ausdrücklich andere Regelungen festgelegt sind.

Mir ist bekannt, dass die Zuwiderhandlung gegen den Inhalt dieser Erklärung einen Täuschungsversuch darstellt, der das Nichtbestehen der Prüfung zur Folge hat und daneben strafrechtlich gem. § 156 StGB verfolgt werden kann. Darüber hinaus ist mir bekannt, dass ich bei schwerwiegender Täuschung exmatrikuliert und mit einer Geldbuße bis zu 50.000 EUR nach der für mich gültigen Rahmenprüfungsordnung belegt werden kann.

Ich erkläre mich damit einverstanden, dass diese Prüfungsleistung zwecks Plagiatsprüfung auf die Server externer Anbieter hochgeladen werden darf. Die Plagiatsprüfung stellt keine Zurverfügungstellung für die Öffentlichkeit dar.

Köln, 24.05.2026

(Ort, Datum)

(Eigenhändige Unterschrift)